大家，我是 Guide。目前本项目有两个场景需要用到 PDF 导出功能：

1. 当用户提交简历生成简历分析建议后，用户可以下载简历分析报告，包含简历各维度评分、优势亮点、改进建议等信息。
2. 当用户完成模拟面试后，用户可以下载面试分析报告，包含面试表现、评分、反馈、改进建议等信息。

技术选型

项目选择 iText 8 作为 PDF 生成库：

```
// build.gradle
implementation "com.itextpdf:itext-core:${libs.versions.itext.get()}"
implementation "com.itextpdf:font-asian:${libs.versions.itext.get()}"
```

除了 iText 8 之外，还有一些其他的 PDF 库可供选择，例如 OpenPDF、Apache PDFBox（我在 Java 优质开源工具类中有详细总结）。

简单对比一下：

对比项	iText 8	OpenPDF	Apache PDFBox
许可证	AGPL（商用需付费）	LGPL/MPL（免费商用）	Apache 2.0（免费商用）
功能丰富度	☆☆☆☆☆	☆☆☆	☆☆☆☆
API 易用性	☆☆☆☆	☆☆☆☆☆	☆☆☆
文档布局能力	强大（表格、多列、样式）	基础	底层控制
中文支持	优秀（font-asian 模块）	良好	一般
维护状态	非常活跃	一般	活跃
学习曲线	中等	低	较高

本项目采用 iText 8 作为 PDF 生成库，主要基于以下考量：

- **功能完备性**：iText 8 提供了业界最完善的 PDF 生成能力，原生支持复杂的文档布局需求，包括多级表格嵌套、灵活的段落样式控制、精确的页面排版等。相比之下，OpenPDF 功能较为基础，Apache PDFBox 则更偏向底层操作，需要开发者自行处理布局逻辑。
- **中文排版支持**：通过 `font-asian` 模块，iText 8 对中日韩文字提供了开箱即用的支持。结合字体嵌入机制，可以确保生成的 PDF 在任何操作系统上都能正确显示中文内容，彻底解决跨平台乱码问题。
- **成熟度与稳定性**：iText 作为 PDF 处理领域的标杆项目，经过二十余年的迭代打磨，API 设计成熟稳定，文档齐全，社区活跃。遇到问题时能够快速找到解决方案，降低了开发和维护成本。
- **企业级特性**：iText 8 支持 PDF/A 归档标准、PDF/UA 无障碍规范、数字签名、表单处理等企业级功能，为项目未来的功能扩展预留了充足空间。

不过，需要注意的是：iText 8 社区版采用 AGPL 许可证。对于开源项目可直接使用；闭源商业项目需要评估是否符合 AGPL 条款，或考虑购买商业许可。如果项目对许可证有严格限制，OpenPDF（LGPL）或 Apache PDFBox（Apache 2.0）是可行的替代方案。

核心实现

PDF 导出服务（`PdfExportService`）

位于 `interview.guide.infrastructure.export` 包，作为基础设施层组件：

```
@Slf4j
@Service
@RequiredArgsConstructor
public class PdfExportService {

    private static final DateTimeFormatter DATE_FORMAT =
        DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
    private static final DeviceRgb HEADER_COLOR = new DeviceRgb(41, 128, 185);
    private static final DeviceRgb SECTION_COLOR = new DeviceRgb(52, 73, 94);

    private final ObjectMapper objectMapper;
    // ...
}
```

中文字体处理

PDF 中文显示是一个常见痛点。项目采用内嵌字体方案，确保跨平台一致性：

```
private PdfFont createChineseFont() {
    try {
        // 使用项目内嵌字体（保证跨平台一致性）
        var fontStream = getClass().getClassLoader()
            .getResourceAsStream("fonts/ZhuqueFangsong-Regular.ttf");
        if (fontStream != null) {
            byte[] fontBytes = fontStream.readAllBytes();
            fontStream.close();
            return PdfFontFactory.createFont(
                fontBytes,
                PdfEncodings.IDENTITY_H,
                EmbeddingStrategy.FORCE_EMBEDDED
            );
        }
        throw new BusinessException(ErrorCode.EXPORT_PDF_FAILED, "字体文件缺失");
    } catch (Exception e) {
        throw new BusinessException(ErrorCode.EXPORT_PDF_FAILED, "创建字体失败");
    }
}
```

关键点：

- 字体文件放在 `src/main/resources/fonts/` 目录
- 使用 `PdfEncodings.IDENTITY_H` 支持 Unicode
- `EmbeddingStrategy.FORCE_EMBEDDED` 强制嵌入字体到 PDF

如果不用内嵌字体方案的话，系统字体检测和降级逻辑会比较复杂且容易出错。

本项目导出 PDF 的第一版采用的就是非内嵌字体方案，结果遇到使用 Windows 系统的朋友出现导出乱码的问题：https://gitee.com/SnailClimb/interview-guide/issues/IDIJU9。

我们选择的字体是朱雀仿宋，这是璇玑造字的开源仿宋字体计划，志在最终提供高质量的、支持多语言的正文仿宋解决方案。项目地址：https://github.com/TrionesType/zhuque。

项目	说明
名称	朱雀仿宋 (Zhuque Fangsong)
许可证	SIL Open Font License 1.1
免费商用	✅ 是
跨平台	✅ Windows / macOS / Linux 通用

特殊字符处理

为避免 emoji 等特殊字符导致渲染问题：

```
private String sanitizeText(String text) {
    if (text == null) return "";
    // 移除可能导致问题的特殊字符（如 emoji）
    return text.replaceAll("[\\p{So}\\p{Cs}]", "").trim();
}
```

面试报告生成

这里以面试报告生成为例进行介绍。

核心方法 `exportInterviewReport` 实现：

```
  public byte[] exportInterviewReport(InterviewSessionEntity session) {
      ByteArrayOutputStream baos = new ByteArrayOutputStream();
      PdfWriter writer = new PdfWriter(baos);
      PdfDocument pdfDoc = new PdfDocument(writer);
      Document document = new Document(pdfDoc);

      // 设置中文字体
      PdfFont font = createChineseFont();
      document.setFont(font);

      // 1. 标题
      Paragraph title = new Paragraph("模拟面试报告")
          .setFontSize(24)
          .setBold()
          .setTextAlignment(TextAlignment.CENTER)
          .setFontColor(HEADER_COLOR);
      document.add(title);

      // 2. 基本信息
      document.add(createSectionTitle("面试信息"));
      document.add(new Paragraph("会话ID: " + session.getSessionId()));
      document.add(new Paragraph("题目数量: " + session.getTotalQuestions()));
      // ...

      // 3. 综合评分（带颜色标识）
      if (session.getOverallScore() != null) {
```

```
            Paragraph scoreP = new Paragraph("总分: " + session.getOverallScore() +
" / 100")
                .setFontSize(18)
                .setBold()
                .setFontColor(getScoreColor(session.getOverallScore()));
            document.add(scoreP);
        }

        // 4. 优势与改进建议（从JSON解析）
        // 5. 问答详情

        document.close();
        return baos.toByteArray();
    }
```

动态评分颜色

根据分数动态显示不同颜色：

```
private DeviceRgb getScoreColor(int score) {
    if (score >= 80) return new DeviceRgb(39, 174, 96);   // 绿色 - 优秀
    if (score >= 60) return new DeviceRgb(241, 196, 15);  // 黄色 - 合格
    return new DeviceRgb(231, 76, 60);                    // 红色 - 需改进
}
```

API 接口设计

Controller 层提供 REST API，这里以面试报告生成为例进行介绍。

```
@GetMapping("/api/interview/sessions/{sessionId}/export")
public ResponseEntity<byte[]> exportInterviewPdf(@PathVariable String sessionId)
{
    try {
        byte[] pdfBytes = historyService.exportInterviewPdf(sessionId);
        String filename = URLEncoder.encode(
            "模拟面试报告_" + sessionId + ".pdf",
            StandardCharsets.UTF_8
        );

        return ResponseEntity.ok()
            .header(HttpHeaders.CONTENT_DISPOSITION,
                "attachment; filename*=UTF-8''" + filename)
            .contentType(MediaType.APPLICATION_PDF)
            .body(pdfBytes);
    } catch (Exception e) {
        return ResponseEntity.internalServerError().build();
    }
}
```

注意点：

- 使用 `filename*=UTF-8''` 格式支持中文文件名
- 返回 `application/pdf` MIME 类型
- 设置 `attachment` 触发浏览器下载

可优化点（作业）

我预留了一些预留的优化方向，作为扩展练习，感兴趣的同学可以尝试实现。

通过这种练习，对个人编码能力的提升是非常有帮助的，Guide 强烈建议编程基础一般的同学都尝试实践一下。

同步转异步

如果报告内容非常多（比如包含大量的 RAG 检索依据），生成 PDF 可能会超过 5 秒。建议补充说明：可以利用项目中已有的 **Redis Stream**，将 PDF 生成任务异步化，生成完成后存储在 S3 (MinIO)，再通过消息/轮询通知用户下载。

HTML 转 PDF

手动编写 `document.add(new Paragraph(...))` 的代码非常繁琐且难以维护（类似于手写 HTML 字符串）。

iText 8 的扩展模块 pdfHTML能够将 HTML/CSS 内容直接转换为高质量的 PDF 文档。它支持大部分 CSS 样式属性，让开发者可以用熟悉的前端技术来设计 PDF 布局，而不必手动编写繁琐的 Java 代码。

比较推荐的方式是结合 Thymeleaf 模板引擎来做。

采用 Thymeleaf + pdfHTML 的开发模式带来以下好处：

- **关注点分离**：HTML 负责结构，CSS 负责样式，Java 只负责数据准备和转换调用。三者各司其职，代码清晰。
- **快速迭代**：调整 PDF 样式只需修改 HTML/CSS 模板，无需重新编译 Java 代码。在开发阶段甚至可以直接用浏览器预览模板效果。
- **复用前端技能**：团队成员可以用熟悉的 HTML/CSS 知识来设计 PDF，降低学习成本。
- **易于维护**：模板文件比 Java 代码更直观，后续接手的开发者能快速理解文档结构。

总结

本项目通过 **iText 8** 构建了一个健壮的 PDF 导出方案，主要用于生成简历分析和面试评估报告。

- **库选择**：放弃了轻量级的 OpenPDF 和底层的 PDFBox，选择 **iText 8**。主要看中其强大的**文档布局能力**（处理表格、多列样式）和完善的 **企业级特性**（PDF/A 支持、稳定性）。不过， iText 8 的 **AGPL 协议**闭源商用需谨慎。
- **解决中文乱码**：采用**内嵌字体**方案（朱雀仿宋），配合 `PdfEncodings.IDENTITY_H`，彻底规避了因操作系统缺失字体导致的渲染失败。
- **稳定性增强**：通过正则过滤（`sanitizeText`）移除 **Emoji** 等可能导致 PDF 渲染崩溃或乱码的特殊字符。